SDE Intern Coding Assignment

AI Resume Screening & Ranking System

Time Box 2–3 hours	**Role** SDE Intern + AI background	**Input** ~50 resumes
Primary Language Python	**Interface** CLI or FastAPI	**Output** Ranked candidates + reasons

Goal: Build a small production-minded system that ingests a folder of resumes, filters candidates against our minimum Python/AI requirements, scores relevant candidates based on engineering and AI project depth, enriches the score using public GitHub activity, and returns a ranked shortlist.

Important: We are evaluating your engineering judgment, not UI polish. A simple, correct, explainable backend is better than an over-engineered application.

1. Problem Statement

You will receive a directory containing approximately 50 candidate resumes. Your application must process the resumes in one run and produce a ranked list of candidates who are relevant for an SDE internship requiring strong Python fundamentals and practical exposure to AI/agentic systems.

Expected pipeline

1. Ingest every resume from the input directory.
2. Extract resume text and useful candidate information such as name, email, skills, projects, and GitHub URL when available.
3. Apply hard eligibility filters before ranking.
4. Score only eligible candidates using the scoring model below.
5. Check public GitHub activity when a GitHub profile is available.
6. Return all candidates with eligibility status, evidence, score breakdown, and final rank.

2. Input Requirements

Assume a folder structure similar to:

```
project/
  resumes/
    candidate_01.pdf
    candidate_02.pdf
    ...
  src/
  README.md
  requirements.txt or pyproject.toml
```

- PDF support is required. DOCX/TXT support is optional and will be considered a bonus.
- Do not assume all resumes have the same layout or section names.
- A malformed or unreadable resume must not crash the complete batch.

3. Eligibility Rules — Hard Filter

Eligibility is intentionally rule-based. The ranking model must not give a high score to an irrelevant profile simply because the resume is well written.

A candidate is eligible only when both conditions are satisfied:

- Python evidence: Python must appear as a genuine skill, project technology, internship/work technology, or implementation language. A JavaScript/Java/React-only profile must be rejected.
- AI/agentic evidence: the resume must show at least one meaningful AI/LLM/RAG/agentic project, framework, or implementation. Examples include LangChain, LangGraph, Google ADK, LlamaIndex, RAG pipelines, embeddings/vector search, tool-calling agents, multi-agent workflows, evaluation pipelines, or equivalent custom implementations.

Do not reject a candidate merely because JavaScript, Java, React, or Next.js is also present. Those skills are acceptable when the candidate also satisfies the Python + AI requirement.

Suggested eligibility output

```
{
  "candidate": "Candidate Name",
  "eligible": false,
  "rejection_reasons": ["No evidence of Python stack", "No AI/agentic project evidence"],
  "matched_skills": ["Java", "React", "Spring Boot"]
}
```

4. Candidate Ranking Score — 100 Points

You may implement the scoring rules deterministically, with an LLM, or with a hybrid approach. The final result must be explainable and include evidence from the resume. The following weighting should be used as the baseline.

Category	Weight	What to reward
AI / Agentic / RAG Project Depth	40	Real AI systems: agents, RAG, tools, retrieval, state, orchestration, evaluation and meaningful business logic.
Python & Backend Engineering	30	Python, FastAPI, async programming, PostgreSQL and Redis. Reward evidence in projects/internships over keyword-only skill lists.
Cloud / Deployment / Full Stack	15	GCP, Docker and deployment. React/Next.js are useful supporting signals when part of an end-to-end system.
GitHub Activity	10	Recent public engineering activity and maintained/relevant repositories. Missing/private GitHub should not fail screening.
Engineering Depth Signals	5	Testing, architecture, caching, queues, observability, concurrency, failure handling and similar non-trivial details.

Project-quality penalties

- Deduct 5–15 points when an “AI project” is only a thin wrapper around an LLM/API call with no meaningful workflow, data processing, retrieval, state management, backend logic, evaluation, or product logic.
- Deduct points for tutorial-style projects that are listed without implementation details or evidence of ownership.

- Do not award full points just because a framework name appears in the skills section. Prefer evidence showing how it was used.
- A candidate with strong Python skills but no meaningful AI project should not rank near the top even if they pass the minimum eligibility rule.

5. GitHub Enrichment

If the resume contains a GitHub profile, enrich the candidate with a small amount of public GitHub information. Keep this implementation lightweight.

- Extract the GitHub username/profile URL from the resume.
- Use the public GitHub API (or equivalent public data) to estimate recent activity.
- Useful signals: recent public events/commits, recently updated repositories, number of maintained public repositories, and repositories relevant to Python/AI.
- Cap GitHub contribution at 10 points. GitHub is an additional positive signal, not a hard eligibility requirement.
- If the GitHub API fails, is rate-limited, or the profile is private/missing, continue screening and record the enrichment failure instead of failing the batch.
- If you use a GitHub token, read it from an environment variable. Never hard-code credentials.

Suggested simple score: 0–5 points for recent activity + 0–5 points for maintained/relevant repositories. You may propose a better method in the README if it remains explainable.

6. AI / LLM Usage

This is an AI-assisted screening assignment, so using an LLM for semantic extraction or project-quality judgment is encouraged. However, the solution should remain predictable and testable.

- Use structured output (for example, a Pydantic schema / JSON schema) when asking an LLM to extract or score resume information.
- Keep hard eligibility checks outside the LLM where possible.
- Ask the model to return evidence or short reasons for important score decisions.
- Keep model/provider-specific code behind a small adapter/function so it can be replaced easily.
- API keys must come from environment variables.
- Your application should still fail gracefully if one model call fails for one resume.

7. Required Output

At minimum, produce a machine-readable JSON or CSV result. JSON is preferred because it can include the score breakdown and reasoning.

```
[
  {
    "rank": 1,
    "candidate_name": "Asha Rao",
    "eligible": true,
    "total_score": 86,
    "score_breakdown": {
      "ai_project_depth": 35,
      "python_backend": 27,
      "cloud_fullstack": 12,
      "github": 8,
      "engineering_depth": 4
    },
    "matched_skills": ["Python", "FastAPI", "PostgreSQL", "LangGraph", "Docker", "GCP"],
    "project_summary": "Built a stateful agentic workflow with retrieval and tool calling.",
    "github_summary": "Recently active; multiple maintained Python repositories.",
    "strengths": ["Strong agentic project", "Async FastAPI backend"],
    "concerns": ["Limited Redis evidence"]
  }
]
```

Minimum final outputs

- Ranked eligible candidates, highest score first.
- Rejected candidates with explicit rejection reasons.
- Matched skills and short evidence-backed project summary.
- Score breakdown for every eligible candidate.
- GitHub enrichment status/summary when a GitHub profile exists.
- Batch summary: total resumes, successfully parsed, eligible, rejected, failed/unreadable.

8. How We Should Be Able to Run It

Choose one of the following. A CLI is sufficient. A small FastAPI interface is welcome if you can complete it cleanly within the time box.

```
# CLI example
python main.py --input ./resumes --output ./output/results.json

# Optional API shape
POST /screen
GET  /results
```

Do not spend time building a frontend. A correct backend pipeline and clean output matter more.

9. Engineering Expectations

- Use clear modules/functions instead of placing the entire solution in one file.
- Handle malformed resumes, missing fields, duplicate files, GitHub API failures, and LLM/API failures reasonably.
- Avoid repeated network calls when data can be cached during the run.
- If you use asynchronous calls for LLM/GitHub enrichment, keep concurrency bounded and understandable.
- Keep configuration (model name, thresholds, weights, API keys) separate from business logic.
- Add at least a few small tests for the most important logic, especially eligibility and scoring.

- Prefer explainability over complicated scoring mathematics.

10. Deliverables

- Source code in a Git repository or ZIP.
- README with setup and run instructions.
- requirements.txt or pyproject.toml.
- .env.example showing required environment variables, without real secrets.
- Generated results.json (or equivalent) for the provided resume set.
- A short README section named “Design Decisions” explaining your filtering strategy, scoring strategy, LLM usage, and GitHub scoring.
- A short README section named “If I Had More Time” listing the next 2–4 improvements you would make.

11. How Your Submission Will Be Evaluated

This rubric evaluates your implementation. It is separate from the 100-point score your program assigns to resumes.

Area	Weight	What we look for
Ingestion + hard filtering correctness	25%	Processes the full folder, handles failures, correctly removes non-Python / non-AI profiles.
Ranking & project-quality logic	25%	Useful ranking, correct priorities, penalties for shallow API-wrapper projects, evidence-backed score.
AI/LLM implementation	15%	Structured output, sensible prompt/adapter design, robust handling of model failures.
Code quality & architecture	15%	Readable modules, configuration separation, maintainability, sensible abstractions.
GitHub enrichment	10%	Useful activity signal, reasonable scoring, graceful rate-limit/error behavior.
Reliability & tests	5%	Important edge cases covered; batch does not fail because of one bad resume.
README & usability	5%	Easy to set up and run; decisions and trade-offs are clearly explained.

12. Scope Guardrails

You have only 2–3 hours. Prioritize the core pipeline. You are not expected to build a production SaaS product.

- No frontend is required.
- No authentication is required.
- No database is required; JSON/CSV output is enough.
- No deployment is required.
- No vector database is required for this assignment.
- Do not spend significant time on visual design.

Bonus ideas (only after core requirements work)

- DOCX resume parsing in addition to PDF.

- Simple FastAPI endpoints around the pipeline.
- Async/bounded-concurrency processing with measurable speed improvement.
- Caching GitHub/LLM results.
- A small HTML/terminal report showing top candidates and score breakdown.
- Unit/integration tests using a few synthetic resumes.

13. Submission Note

We do not expect a perfect model or perfect resume parsing in 2–3 hours. We care about whether you can convert an ambiguous real-world requirement into a reliable engineering pipeline, make sensible trade-offs, write understandable code, and explain why the system ranked or rejected a candidate.

Please submit the implementation exactly as you would hand it to another engineer to run and review.